

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tarea - TIA-06 (Unidad 3 - Tarea 2)

Consultas avanzadas con agrupamientos (DML)

Integración de BD relacionales con NoSQL en escenarios de Big Data e IoT

- Tarea en Equipo
- Peso: 20%
- Práctica. Caso de Estudio (Red Social PAscualina)
- DML - Manipulación de Datos - Agrupamientos - Funciones de Agregación
- Proyecto de Aula

MIEMBROS DEL EQUIPO:

- Líder: Brayan Zabdiel Leon Mendoza
- Miembro: Kely Andrea Barrientos Pino – Lesly Tatiana Tabares Muñoz – Tomas Henao Cardona

Descripción de la Tarea

1. Contextualización

Esta es la última fase del Proyecto de Aula. En este momento, se dispone de:

- Un modelo físico madurado. Una base de datos física mal relacionada (referencias entre tablas)
- Establecimiento de controles para evitar inconsistencias y embasuramiento de datos después de aplicar correctivos al revisar las propiedades ACID
- Se ha logrado una base de datos coherente y correcta estructuralmente
- Protegida para evitar inconsistencias durante procesos de actualización
- Amigable y flexible para realizar consultas con información de calidad

En anteriores tareas, el reto consistió en tomar el modelo físico previamente construido para el sistema de Red Social Pascualina y transformarlo en un modelo completamente funcional, implementado en un SGBD, para poder realizar operaciones de inserción, modificación, eliminación y consultas a través del lenguaje de manipulación de datos (DML). Adicionalmente, se realizaron pruebas para verificar el cumplimiento de las propiedades ACID.

La fase final que consolida el proyecto de Aula (PA) requiere:

- Construcción de consultas con datos parametrizados
- Construcción de consultas avanzadas con agrupamientos y funciones de agregación
- Responder a un conjunto de consultas solicitadas por el Consejo de la Red Social Pascualina
- Inserción, modificación, eliminación y consulta de datos semi estructurados tipo JSON y JSONB; además de explicar la diferencia entre estos.
- Analizar escenarios de Big Data e IoT para comprender el impacto del manejo de datos masivos.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Comprendan la eficiencia de las consultas con parámetros y su reutilización
- Desarrollen la habilidad de construir consultas, a través de los agrupamientos de datos, que respondan a planteamientos organizacionales de importancia
- Manipulen correctamente datos semi estructurados como JSON y JSONB
- Adquieran nuevas herramientas para evaluar el rendimiento de las bases de datos en escenarios de manejo de grandes volúmenes de datos.

3. Instrucciones de realización de la tarea

Experimentar en el SGBD establecido (PostgreSQL), de acuerdo con las necesidades de la red social Pascualina. Luego, utilizando una herramienta gráfica (pgAdmin4), se enfocarán en la correcta manipulación del modelo, prestando especial atención al correcto funcionamiento de las instrucciones de inserción, actualización, eliminación y consulta de datos semi-estructurados (JSON/JSONB), consultas con parámetros (PREPARE, EXECUTE) basadas en vistas (VIEW), consultas con agrupamientos y funciones de agregación (WHERE, GROUP BY HAVING), análisis de rendimiento del motor de base de datos (escenarios Bi Data e IoT) a través de la medición de la velocidad de preparación y ejecución de consultas (EXPLAIN ANALYZE).

Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables clave:

- Un informe técnico que justifique sus decisiones.
- El conjunto de scripts de manipulación solicitados en el formato de plantilla suministrado (ESTE MISMO).
- Análisis de los resultados, conclusiones generales y reflexiones individuales
- Un video corto donde sustenten el trabajo realizado y las conclusiones individuales que reflejen su aprendizaje personal. **DEBE MOSTRARSE EL ESTUDIANTE E IDENTIFICARSE CON SU NOMBRE. DEBE MOSTRAR CÓDIGO EN EJECUCIÓN -> [Mostrar ejecución de scripts de manipulación en el SGBD.](#)**

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DML que permiten recoger información estadística de relevancia así como los “INSIGHTS” relevantes para una organización a través de consultas directas o parametrizadas validando su rendimiento. Adicionalmente, tendrá la oportunidad de experimentar escenarios simulados de “ingesta” masiva de datos en el contexto de Big Data e IoT.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

4. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura: **Carpeta raíz:**

Tarea 6. En esta carpeta estarán tres carpetas con los entregables: Informe, Scripts y Video.

Carpetas	Contenido requerido	Observaciones
/tarea6	Todas las subcarpetas y archivos de la tarea.	No aplica
tarea6/Informe	Informe detallado (plantilla suministrada)	.pdf
tarea6/Scripts	Todos los Scripts solicitados	.sql
tarea6/video	Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y las conclusiones individuales.	Enlace a YouTube

Estos son las plantillas de formato a utilizar OBLIGATORIAMENTE. Solamente debe cambiar la “X” por el Equipo de estudiantes

Evidencias de aprendizaje evaluadas

Evidencia	Habilidad evaluada
Informe con el paso a paso de la propuesta de solución, teniendo en cuenta: - Operaciones de consultas con parámetros a través de vistas (PREPARE, EXECUTE) - Operaciones de manipulación de datos JSON - Operaciones de manipulación de datos JSONB - Comparación del rendimiento de los datos semi estructurados: JSON y JSONB - Operaciones de consulta de información mediante agrupamientos y funciones de agregación (WHERE, GROUP BY, HAVING) - Operaciones de manipulación de datos para experimentar el rendimiento de bases de datos en el contexto de Big Data e IoT (EXPLAIN ANALYZE) - El informe debe referenciar los scripts DML y explicar su estructura. - Conclusiones generales individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso.	Capacidad de análisis y redacción técnica Reflexión crítica y aprendizaje personal.
Tipos de dato SQL y NoSQL para big data e IoT con justificación técnica del uso de tipos de datos adecuados para escenarios de big data e IoT Ejemplo: campo JSON para las características de salud de un usuario de la Red Social	Comprensión y utilización de tipos de datos que representen una relación con soluciones big data e IoT .
Modelo Analítico implementado a través de operaciones DML - Agrupamiento: - Campos correctamente seleccionados - Relaciones adecuadas a cada planteamiento (JOIN). - Agrupamiento correcto de las consultas para obtener la información solicitada - Utilización correcta de las funciones de agregación según el planteamiento - Evaluación adecuada del “performance” de la base de datos en escenarios simulados de Big Data e IoT. - Uso correcto las reglas, nomenclatura y estándares para BD físicas.	Comprensión del Lenguaje de Manipulación de Datos (DML) de Agrupamiento
Participación en Foro de discusión y seguimiento de tareas	Comunicación asíncrona y trabajo en equipo

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Evidencia	Habilidad evaluada
Video de sustentación (presentarse con imagen y nombre. Mostrar y explicar código en ejecución)	Comunicación oral y trabajo en equipo
Específicos	Competencia en gestión de versiones y organización digital.

ATENCIÓN: Analicen los criterios de evaluación y verifiquen su cumplimiento así como la ponderación de los ítems en la RÚBRICA.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Informe con resultados

Equipo "X"

(TIA-6: Unidad 3 - Tarea 2)

1.- Vista (VIEW) agrupamientos (GROUP BY) parametrizada (PREPARE)

The screenshot shows a PostgreSQL IDE interface. On the left, a tree view displays the database structure, including 'Servers (1)' with 'PostgreSQL 18', and 'Databases (4)' with 'Red_Social'. The 'Red_Social' database is expanded, showing 'Casts', 'Catalogs', 'Event Trigger', 'Extensions', 'Foreign Data', 'Languages', 'Publications', 'Schemas', and 'Subscriptions'. The 'public' schema is selected, showing 'nueva', 'Casts', 'Catalogs', 'Event Trigger', 'Extensions', 'Foreign Data', 'Languages', 'Publications', 'Schemas (1)', and 'Aggregates'.

The main window displays a query editor with the following SQL code:

```
66
67 -- EXECUTE 2:
68 EXECUTE analisis_actividad_eventos('programado', 1); -- Eventos programados con más de 1 participante inscrito
69
70 -- EXECUTE 3:
71 EXECUTE analisis_actividad_eventos('en_curso', 2); -- Eventos en curso con más de 2 participantes activos
72
73 -- Analisis del plan de ejecución
74 EXPLAIN ANALYZE
75 EXECUTE analisis_actividad_eventos('finalizado', 5);
```

Below the query editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' for the executed query. The plan details are as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Sort	76.56..76.65	36	228	0.960..0.965	9.00	1
2	Sort Key: (count(up.id_usuario)) DESC, (count(DISTINCT e.id_evento)) DESC						
3	Sort Method: quicksort Memory: 26kB						
4	Buffers: shared hit=27						
5	GroupAggregate	72.48..75.63	36	228	0.897..0.948	9.00	1
6	Group Key: (((uc.nombre)::text ' '::text) (uc.apellido)::text), uc.correo, te.nombre_tipo						

The status bar at the bottom indicates 'Total rows: 53' and 'Query complete 00:00:00.206'. The page number is 'Page No: 1 of 1'.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2.- Script de consultas *agrupamientos* y funciones de agregación (*SELECT-JOIN-WHERE-GROUP BY-HAVING*).

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is for 'Consulta #1: Grupos' and is as follows:

```
-- Consulta #1: Grupos
SELECT
    g.id_grupo AS "ID Grupo",
    g.nombre AS "Nombre del Grupo",
    g.tipo_grupo AS "Tipo de Grupo",
    g.fecha_creacion AS "Fecha de Creación",
    u.nombre || ' ' || u.apellido AS "Nombre Completo Creador",
    COUNT(ug.id_usuario) AS "Total de Miembros"
FROM grupos g
INNER JOIN usuarios u ON u.id_usuario = g.id_administrador
LEFT JOIN usuario_grupo ug ON ug.id_grupo = g.id_grupo
GROUP BY
    g.id_grupo,
    g.nombre,
    g.tipo_grupo,
    g.fecha_creacion,
    u.nombre,
    u.apellido
ORDER BY g.nombre ASC;
```

The results are displayed in a table with the following data:

ID Grupo	Nombre del Grupo	Tipo de Grupo	Fecha de Creación	Nombre Completo Creador	Total de Miembros
8	Alumni Electrónica	egresados	2026-01-30	Marcela Suárez Lozano	13
5	Apoyo Académico Cálculo I	apoyo_academico	2026-01-03	Camilo Ruiz Ríos	17
10	Apoyo Académico Estadística	apoyo_academico	2026-05-02	Rodrigo Cardona Correa	11
7	Club de Robótica	proyecto	2026-03-19	Camilo Ramírez Cifuentes	13

Successfully run. Total query runtime: 78 msec. 10 rows affected.

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is for 'Consulta #3: Tipos de Servicio' and is as follows:

```
-- Consulta #3: Tipos de Servicio
SELECT
    ts.id_tipo_servicio AS "ID Tipo Servicio",
    ts.nombre_tipo AS "Nombre Tipo Servicio",
    COUNT(tr.id_usuario_cliente) AS "Total Usuarios que lo Consumieron",
FROM eventos e
INNER JOIN tipos_evento te ON te.id_tipo_evento = e.id_tipo_evento
INNER JOIN usuarios u ON u.id_usuario = e.id_creador
LEFT JOIN evento_usuarios eu ON eu.id_evento = e.id_evento
GROUP BY
    te.id_tipo_evento,
    te.nombre_tipo,
    u.nombre,
    u.apellido,
    e.id_evento,
    e.nombre_evento,
    e.fecha_hora_inicio,
    e.estado
ORDER BY e.fecha_hora_inicio DESC;
```

The results are displayed in a table with the following data:

ID Tipo Evento	Tipo de Evento	Nombre Promotor	Descripción del Evento	Fecha de Inicio	Estado del Evento	Total Usuarios Inscritos
11	concierto	Harold Mendoza Forero	Congreso de Automatización Industrial	2026-05-15 04:42:43	en_curso	7
1	congreso	Valentina Mendoza Alzate	Simposio de Manufactura Avanzada	2026-05-11 06:40:33	cancelado	8
10	foro	Harold Bedoya Arenas	Feria Laboral Pascual Bravo 2026	2026-05-10 05:00:06	finalizado	8
1	congreso	Felipe Aguilar Holguín	Concierto Cultural de Bienvenida	2026-05-09 16:57:03	finalizado	8

Successfully run. Total query runtime: 81 msec. 50 rows affected.

Institución Universitaria Pascual Bravo

Facultad de Ingeniería - Departamento de Sistemas Digitales

Base de Datos I (SD1006)

The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'amen/postgres@PostgreSQL 18' database. The query is a complex SELECT statement with multiple joins and filters. The results are displayed in a table with 5 columns: ID Tipo Servicio, Nombre Tipo Servicio, Total Usuarios que lo Consumieron, and Usuarios Únicos Consumidores. The query is successfully run, with a runtime of 124 msec and 19 rows affected.

```
SELECT
    ts.id_tipo_servicio AS "ID Tipo Servicio",
    ts.nombre_tipo AS "Nombre Tipo Servicio",
    COUNT(tr.id_usuario_cliente) AS "Total Usuarios que lo Consumieron",
    COUNT(DISTINCT tr.id_usuario_cliente) AS "Usuarios Únicos Consumidores"
FROM transacciones_servicio tr
INNER JOIN servicios s ON s.id_servicio = tr.id_servicio
INNER JOIN tipos_servicio ts ON ts.id_tipo_servicio = s.id_tipo_servicio
WHERE tr.fecha_transaccion BETWEEN '2026-01-01 00:00:00'
    AND '2026-03-31 23:59:59'
GROUP BY
    ts.id_tipo_servicio,
    ts.nombre_tipo
ORDER BY COUNT(tr.id_usuario_cliente) DESC;
```

ID Tipo Servicio	Nombre Tipo Servicio	Total Usuarios que lo Consumieron	Usuarios Únicos Consumidores
1	diseño_grafico	20	20
2	traduccion_document...	12	12
3	marketing_digital	12	12
4	produccion_musical	11	11

The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'amen/postgres@PostgreSQL 18' database. The query is a complex SELECT statement with multiple joins and filters. The results are displayed in a table with 6 columns: Tipo de Producto, Total Productos Vendidos, Total Vendedores Únicos, Monto Total de Ventas (COP), Precio Promedio (COP), and Precio Máximo (COP). The query is successfully run, with a runtime of 77 msec and 14 rows affected.

```
SELECT
    tp.nombre_tipo AS "Tipo de Producto",
    COUNT(p.id_producto) AS "Total Productos Vendidos",
    COUNT(DISTINCT p.id_usuario) AS "Total Vendedores Únicos",
    SUM(p.precio) AS "Monto Total de Ventas (COP)",
    ROUND(AVG(p.precio), 2) AS "Precio Promedio (COP)",
    MAX(p.precio) AS "Precio Máximo (COP)",
    MIN(p.precio) AS "Precio Mínimo (COP)"
FROM productos p
INNER JOIN tipos_producto tp ON tp.id_tipo_producto = p.id_tipo_producto
WHERE p.estado_oferta = 'vendido'
GROUP BY
    tp.nombre_tipo
ORDER BY SUM(p.precio) DESC;
```

Tipo de Producto	Total Productos Vendidos	Total Vendedores Únicos	Monto Total de Ventas (COP)	Precio Promedio (COP)	Precio Máximo (COP)	Precio Mínimo (COP)
uniforme_institucional	3	3	5745710	1915236.67	2596787	1314943
calzado_deportivo	2	2	4012874	2006437.00	2205832	1807042
suplemento_alimenticio	2	2	3820222	1910111.00	3088686	731536
electrodomestico	2	2	3599311	1799655.50		

Institución Universitaria Pascual Bravo

Facultad de Ingeniería - Departamento de Sistemas Digitales

Base de Datos I (SD1006)

pgAdmin 4

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (2)
 - amen
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views

Query

```
--
-- Consulta #6: LIBRE DE PLANTEAMIENTO
--
SELECT
    pub.id_publicacion AS "ID Publicación",
    pub.tipo_contenido AS "Tipo de Contenido",
    pub.fecha_publicacion AS "Fecha de Publicación",
    u.nombre || ' ' || u.apellido AS "Nombre Completo Autor",
    pf.alias_usuario AS "Alias del Autor",
    pf.semestre_cursado AS "Semestre del Autor",
    COUNT(c.id_comentario) AS "Total de Comentarios"
FROM
    publicaciones pub
INNER JOIN usuarios u ON u.id_usuario = pub.id_usuario
INNER JOIN perfil pf ON pf.id_usuario = u.id_usuario
INNER JOIN comentarios c ON c.id_publicacion = pub.id_publicacion
WHERE pub.fecha_publicacion >= '2026-01-01'
GROUP BY
    pub.id_publicacion,
    pub.tipo_contenido,
    pub.fecha_publicacion,
    u.nombre,
    u.apellido,
    pf.alias_usuario,
    pf.semestre_cursado
HAVING COUNT(c.id_comentario) > 2
ORDER BY COUNT(c.id_comentario) DESC;
```

Data Output

ID Publicación integer	Tipo de Contenido character varying (20)	Fecha de Publicación timestamp without time zone	Nombre Completo Autor text	Alias del Autor character varying (50)	Semestre del Autor integer	Total de Comentarios bigint
1	25 texto	2026-03-19 07:01:59	Pablo Ruiz Holguín	pascal0245itpb	3	5
2	1 enlace	2026-02-14 23:57:24	Cindy Gómez Correa	tech0054star	[null]	4
3	62 imagen	2026-01-13 07:07:41	Adriana Mendoza Correa	dev0447medellin	3	4
4	61 imagen	2026-05-07 04:14:40	Fernando Reyes González	bravo0133123	9	

Total rows: 20 Query complete 00:00:00.070

Showing rows: 1 to 20 Page No: 1 of 1

Successfully run. Total query runtime: 70 msec. 20 rows affected.

CRLF Ln 152, Col 1

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3.- Script de manipulación de inserción, actualización, eliminación y consulta de una dato JSON

The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Displays the database structure. The 'perfil' table is selected, showing columns: id_perfil (integer), id_usuario (integer), alias_usuario (character), biografia (text), foto_url (character varying), semestre_cursado (integer), fecha_creacion (timestamp), and datos_dispositivo (json).
- Query Editor:** Contains the following SQL script:

```
88 ALTER TABLE perfil
89 ADD COLUMN IF NOT EXISTS datos_dispositivo JSON;
90
91 --
92 -- 1.- Inserción del dato semi estructurado en nuevo campo JSON
93 --
94
95 UPDATE perfil
96 SET datos_dispositivo = '{
97     "dispositivo": {
98         "marca": "Samsung",
99         "modelo": "Galaxy A54",
100         "sistema_operativo": "Android",
101         "version_so": "13.0"
102     },
103     "aplicacion": {
104         "nombre": "Red Pascualina",
105         "version": "2.4.1",
106         "idioma": "es-CO"
107     },
108     "acceso": {
109         "ultimo_acceso": "2026-05-20",
110         "..."
111     }
112 }'
```
- Data Output:** Shows the result of the query. The first row indicates that the column 'datos_dispositivo' was added to the 'perfil' table. The second row shows the updated record for 'perfil' with the new JSON data.

id_perfil (PK) integer	datos_acceso json
1	{ "hora_acceso": "08:35:00", "tipo_conexion": "WiFi", "ultimo_acceso": "2026-05-20", ... }

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

4.- Script de manipulación de inserción, actualización, eliminación y consulta de una dato JSONB

The screenshot shows the pgAdmin 4 interface with a SQL script being executed. The script adds a new JSONB column to the 'perfil' table and updates a record with semi-structured data.

```
215
216 ALTER TABLE perfil
217     ADD COLUMN IF NOT EXISTS datos_dispositivo_b JSONB;
218
219
220 --
221 -- 1.- Inserción del dato semi estructurado en nuevo campo
222 --
223
224 UPDATE perfil
225 SET  datos_dispositivo_b = '{
226     "dispositivo": {
227         "marca": "Samsung",
228         "modelo": "Galaxy A54",
229         "sistema_operativo": "Android",
230         "version_so": "13.0"
231     },
232     "aplicacion": {
233         "nombre": "Red Pascualina",
234         "version": "2.4.1",
235         "idioma": "es-CO"
236     },
237     "acceso": {
238         "ultimo_acceso": "2026-05-28"
239     }
240 }';
```

The Data Output tab shows the result of the update query:

id_perfil [PK] integer	datos_dispositivo_b jsonb
1	('aplicacion': ('idioma': 'es-CO', 'nombre': 'Red Pascualina', 'version': '2.4.1'), 'dispositivo': ('marca': 'Apple', 'modelo': 'iPhone 14', 'version_so': '17.2', 'sistema_operativo': ...

Total rows: 1 Query complete 00:00:00.100 CRLF Ln 361, Col 13

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

JSON

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure with 'public' schema expanded, showing 'Tables (2)'. The main pane shows a SQL query in the 'Query' tab:

```
16      'edad', 20,  
17      'ciudad', 'Girardota',  
18      'activo', true  
19    )  
20  FROM generate_series(1,500);  
21  
22  EXPLAIN ANALYZE  
23  SELECT *  
24  FROM datos_json;  
25  
26  EXPLAIN ANALYZE  
27  INSERT INTO datos_jsonb (informacion)  
28  SELECT  
29    jsonb_build_object(  
30      'nombre', 'Daniel',  
31      'edad', 20,  
32      'ciudad', 'Girardota',  
33      'activo', true  
34    )  
35  FROM generate_series(1,500);
```

The 'Data Output' tab shows the query plan for the first query (lines 16-20):

QUERY PLAN
1 Insert on datos_json (cost=0.00..8.75 rows=0 width=0) (actual time=20.135..20.136 rows=0.00 loops=1)
2 Buffers: shared hit=1632 read=1 dirtied=12 written=13
3 -> Function Scan on generate_series (cost=0.00..8.75 rows=500 width=36) (actual time=0.756..4.127 rows=500.00 loops=1)
4 Buffers: shared hit=513
5 Planning Time: 1.034 ms
6 Execution Time: 21.504 ms

JSONB

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure with 'public' schema expanded, showing 'Tables (2)'. The main pane shows a SQL query in the 'Query' tab:

```
27  INSERT INTO datos_jsonb (informacion)  
28  SELECT  
29    jsonb_build_object(  
30      'nombre', 'Daniel',  
31      'edad', 20,  
32      'ciudad', 'Girardota',  
33      'activo', true  
34    )  
35  FROM generate_series(1,500);  
36  
37  EXPLAIN ANALYZE  
38  SELECT *  
39  FROM datos_jsonb;
```

The 'Data Output' tab shows the query plan for the second query (lines 27-35):

QUERY PLAN
1 Seq Scan on datos_jsonb (cost=0.00..13.00 rows=500 width=88) (actual time=0.036..0.128 rows=500.00 loops=1)
2 Buffers: shared hit=8
3 Planning:
4 Buffers: shared hit=17
5 Planning Time: 3.237 ms
6 Execution Time: 0.219 ms

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tabla “perfil” (500 operaciones de lectura/escritura)

Dato Semi-Estrcuturado	Tipo	Escritura tiempo ms		Lectura tiempo ms	
<i>Registro de los 500 usuarios en JSON</i> <i>Registro de los 500 usuarios en JSONB</i>	JSON	Preparación	Ejecución	Preparación	Ejecución
		1.034	21.504	5.554	0.201
	JSONB	Preparación	Ejecución	Preparación	Ejecución
		0.124	18.054	3.237	0.219
Diferencias		JSONB presentó menor tiempo de preparación en comparación con JSON, mostrando mayor optimización al planificar la inserción de datos.	JSONB ejecuto la inserción más rapido que los JSON.	JSONB presentó mejor tiempo de preparación en la lectura.	JSON presentó una lectura ligeramente más rápida en comparación con JSONB en la prueba realizada.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

6.- Script de simulación de operaciones masivas de datos (Big Data / IoT)

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including a table named 'usuario_test' with 9 columns. The main query editor contains the following SQL script:

```
SELECT ROUND(pg_total_relation_size('usuario_test') / 1024.0 / 1024.0, 2) AS tamaño_mb;

-- Lectura (SELECT con extracción de campos JSON)
EXPLAIN ANALYZE
SELECT id,
       codigo_usuario,
       datos_salud --> 'grupo_sanguineo' AS grupo_sanguineo,
       datos_salud --> 'temperatura_corporal' AS temperatura,
       datos_salud --> 'presion_sanguinea' --> 'sistolica' AS presion_sistolica
FROM usuario_test LIMIT 100;

TRUNCATE TABLE usuario_test RESTART IDENTITY;

SELECT ROUND(pg_total_relation_size('usuario_test') / 1024.0 / 1024.0, 2) AS tamaño_mb;
```

The Query History tab shows the execution plan for the query. The plan includes the following steps:

Step	Operation	Cost	Time
1	Limit	(cost=0.00..6.20 rows=100 width=112)	(actual time=0.032..0.758 rows=100.00 loops=1)
2	Buffers: shared hit=6		
3	Seq Scan on usuario_test	(cost=0.00..62.00 rows=1000 width=112)	(actual time=0.030..0.749 rows=100.00 loops=1)
4	Buffers: shared hit=6		
5	Planning		
6	Buffers: shared hit=12		
7	Planning Time	0.379 ms	
8	Execution Time	0.783 ms	

The Data Output tab shows the results of the query, including the size of the table and the execution plan.

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including a table named 'usuario_test' with 9 columns. The main query editor contains the following SQL script:

```
EXPLAIN ANALYZE SELECT generar_usuarios_test(10000000);

SELECT ROUND(pg_total_relation_size('usuario_test') / 1024.0 / 1024.0, 2) AS tamaño_mb;

EXPLAIN ANALYZE
SELECT id,
       codigo_usuario,
       datos_salud --> 'grupo_sanguineo' AS grupo_sanguineo,
       datos_salud --> 'temperatura_corporal' AS temperatura,
       datos_salud --> 'presion_sanguinea' --> 'sistolica' AS presion_sistolica
FROM usuario_test LIMIT 100;

TRUNCATE TABLE usuario_test RESTART IDENTITY;

SELECT ROUND(pg_total_relation_size('usuario_test') / 1024.0 / 1024.0, 2) AS tamaño_mb;
```

The Query History tab shows the execution plan for the query. The plan includes the following steps:

Step	Operation	Cost	Time
1	Limit	(cost=0.00..19.16 rows=100 width=112)	(actual time=0.042..0.982 rows=100.00 loops=1)
2	Buffers: shared hit=6		
3	Seq Scan on usuario_test	(cost=0.00..538409.46 rows=2809623 width=112)	(actual time=0.041..0.970 rows=100.00 loops=1)
4	Buffers: shared hit=6		
5	Planning Time	0.119 ms	
6	Execution Time	1.020 ms	

The Data Output tab shows the results of the query, including the size of the table and the execution plan.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tabla “usuario_test”

Simulación	Registros	Escritura tiempo ms		Lectura tiempo ms		Tamaño
#	#	Preparación	Ejecución	Preparación	Ejecución	MB
1	1.000	0.034	22.520	0.379	0.783	0.40
2	10.000	0.032	319.864	0.099	0.895	3.89
3	100.000	0.029	3395.735	0.475	1.055	39.23
4	1.000.000	0.028	26572.927	0.097	2.059	397.89
5	10.000.000	0.029	238754.614	0.694	1.046	4006.31

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

7.- Análisis de resultados

El desarrollo de esta fase del proyecto para la Red Social nos permitió evaluar a fondo el comportamiento de PostgreSQL cuando se enfrenta a diferentes estrategias de manipulación, estructuración y escalabilidad de datos. En primer lugar, mediante las operaciones de manipulación de datos (DML) con agrupamientos y funciones de agregación, logramos consolidar métricas de negocio reales, demostrando que herramientas como GROUP BY y HAVING son indispensables para transformar registros individuales en resúmenes estadísticos útiles para la toma de decisiones. Asimismo, la experiencia con el lenguaje de consulta estructurado, implementando vistas combinadas con consultas parametrizadas mediante comandos PREPARE y EXECUTE, demostró cómo se puede optimizar el desarrollo de software. Esto evita tener que reescribir código repetitivo y permite reutilizar una misma estructura lógica para diferentes necesidades del campus con tiempos de respuesta óptimos.

Por otra parte, un punto crítico del trabajo fue el análisis del rendimiento en el almacenamiento de datos semiestructurados. Al comparar los formatos JSON y JSONB mediante la sentencia EXPLAIN ANALYZE, pudimos evidenciar un contraste muy importante: el formato JSON guarda el texto plano tal cual llega, lo que acelera la velocidad de inserción, pero ralentiza las consultas; por el contrario, JSONB descompone el objeto en un formato binario que, aunque toma un poco más de tiempo al guardarse, permite lecturas ultra rápidas y admite la creación de índices inversos generalizados (GIN), algo fundamental para hacer búsquedas eficientes en bases de datos modernas.

Finalmente, las simulaciones masivas en la tabla de pruebas (desde mil hasta millones de registros) nos abrieron los ojos sobre las implicaciones reales de manejar grandes volúmenes de datos e Internet de las cosas (IoT). Al procesar ráfagas masivas de datos simulados de salud y telemetría, confirmamos que el verdadero reto de una arquitectura de Big Data no es solo la capacidad de almacenamiento en disco, sino el impacto que esto genera en la memoria RAM y en los tiempos de respuesta del servidor. Dominar estas herramientas híbridas en PostgreSQL nos demostró que es totalmente posible mantener la seguridad y el orden del modelo relacional tradicional, estando al mismo tiempo preparados para la velocidad y la flexibilidad que exigen las aplicaciones modernas en el mundo real.

8.- Reflexiones y conclusiones individuales

Brayan Leon: Cuando empezamos el curso, no tenía idea de lo que realmente implicaba diseñar una base de datos. Pensaba que era solo crear tablas y guardar información, pero con el desarrollo del proyecto de ByteCampus me di cuenta de que es mucho más profundo. Aprendí que el modelo de datos no es un simple requisito, sino el esqueleto que sostiene toda la aplicación. Si desde el principio no defines bien las entidades, las relaciones y las cardinalidades, después todo se vuelve un caos lleno de redundancias y errores. Normalizar tablas hasta llegar a 3FN, aplicar restricciones con claves foráneas y hasta usar JSON para darle flexibilidad al sistema me hizo entender cómo se construye algo que realmente funcione en el mundo real.

El trabajo en equipo fue clave. Entre los cuatro nos repartimos los scripts, pero también nos tocó sentarnos a debatir cómo conectar las tablas con las llaves foráneas y por qué ciertas operaciones con DELETE o UPDATE podían romper la integridad de los datos. Al principio nos costó entender las propiedades ACID, sobre todo el aislamiento y la durabilidad, pero después de hacer pruebas con BEGIN y COMMIT, vimos que sin esas reglas cualquier fallo del sistema dejaría la información hecha un desastre. También nos enfrentamos a desafíos como

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

aprender a usar `pg_dump` para respaldos (backups) o crear vistas que simplificarán consultas complejas con varios JOIN. Cada error en la sintaxis o en el tipo de dato nos obligaba a revisar y corregir, y eso terminó siendo la mejor forma de aprender.

Este curso me dejó una enseñanza enorme para mi futuro profesional. Ahora sé que una base de datos bien diseñada no solo almacena, sino que protege la información y permite que una plataforma crezca sin volverse insostenible. Aunque al principio me sentí abrumado con tanto comando y regla, hoy valoro cada tropiezo porque me hizo más cuidadoso y analítico. Sin duda, lo aprendido aquí lo voy a usar en cualquier proyecto que toque más adelante, desde una red social hasta un sistema de ventas. Y lo mejor es que ya no le tengo miedo a meterme de lleno en el código.

Lesly Tabares: El desarrollo de las diferentes entregas del proyecto red social me permitió comprender de manera progresiva cómo se construye, administra y optimiza una base de datos en PostgreSQL, pasando desde el diseño inicial hasta simulaciones masivas de datos similares a escenarios reales de Big Data e IoT.

Trabajando en la construcción de la base de datos desde el modelo entidad-relación hacia la implementación física en PostgreSQL. Esto fue fundamental porque me permitió comprender la importancia de definir correctamente las tablas, tipos de datos, claves primarias y claves foráneas para garantizar la integridad de la información. Además, se logró entender que una base de datos bien estructurada es la base principal para que cualquier sistema funcione correctamente y pueda crecer sin presentar inconsistencias o problemas de rendimiento.

Profundizar en los diferentes sistemas gestores de bases de datos como PostgreSQL, MySQL y SQL Server, realizando procesos de migración y comparación entre ellos. Esto permitió identificar diferencias importantes en sintaxis, tipos de datos y funcionamiento interno de cada SGBD. También trabajamos con datos semiestructurados como lo es JSON Y JSONB comprendiendo cómo PostgreSQL permite combinar características relacionales con estructuras más flexibles.

En la tarea 5 realizamos procesos fundamentales para la administración de la base de datos, mediante scripts CREATE y el poblamiento de información utilizando scripts INSERT. Además, se realizaron operaciones UPDATE y DELETE para modificar y eliminar información de manera controlada, comprendiendo la importancia de mantener la integridad de los datos durante la manipulación de registros. También se implementaron consultas SELECT y vistas VIEW para facilitar la recuperación organizada de información.

Finalmente se realizó un análisis de rendimiento y procesamiento masivo de información. Inicialmente se realizaron pruebas comparativas entre JSON y JSONB mediante operaciones de escritura y lectura sobre 500 registros, utilizando EXPLAIN ANALYZE para medir tiempos de preparación y ejecución. Gracias a estas pruebas se logró identificar que JSON presenta ventajas en ciertas operaciones de escritura debido a su almacenamiento textual, mientras que JSONB ofrece una mejor optimización para consultas, búsquedas y procesamiento interno de información gracias a su formato binario.

Todas las entregas estuvieron completamente conectadas entre sí y permitieron comprender el ciclo completo de vida de una base de datos moderna, desde su diseño inicial hasta pruebas avanzadas de rendimiento y escalabilidad.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tomas Henao: Hacer esta segunda parte del trabajo, metiéndonos de lleno con los agrupamientos y las funciones estadísticas en pgAdmin 4, nos sirvió demasiado para entender cómo se transforma una base de datos en verdadera información de valor. En la fase anterior solo aprendimos a buscar datos sueltos, pero con este taller vimos que el verdadero poder de la red social "Pascualina" está en procesar la información en masa. Al empezar a tirar códigos con COUNT, SUM y AVG para contar los miembros de los grupos o sumar las ventas de los productos, caímos en la cuenta de que la analítica es la que permite cambiar miles de filas ignoradas en estadísticas reales que le sirven a los jefes para tomar decisiones en el campus.

Por otro lado, pelear con el GROUP BY y hacer los cruces de las tablas nos obligó a entender con total claridad cómo está amarrado todo el modelo físico. Lograr que PostgreSQL agrupara de forma limpia los aforos de los eventos o las transacciones de servicios sin que nos sacara errores, nos demostró la importancia de tener las llaves primarias y foráneas bien configuradas desde el principio. Aprendimos que si el diseño de tu equipo no está bien normalizado, los cálculos globales se dañan por completo.

Finalmente, aplicar el HAVING en el insight libre nos abrió la mente como desarrolladores. Entender la diferencia técnica entre filtrar filas simples con el WHERE y filtrar estadísticas ya agrupadas con el HAVING nos dio el control total para encontrar datos avanzados, como las publicaciones que se volvieron virales. En conclusión, esta entrega nos dejó el aprendizaje para dejar de ver la base de datos como un simple archivador y empezar a usarla como una herramienta estratégica para el negocio.

Kely Barrientos: Con la realización de este trabajo entendí mucho mejor cómo funciona una consulta compleja en bases de datos y por qué se usan herramientas como vistas, PREPARE y EXECUTE. Al principio para construir una consulta no veía necesario hacer tantos pasos y pensaba que sería más fácil escribir la consulta directamente cada vez que se necesitara, pero luego mientras iba desarrollando el ejercicio, empecé a entender la lógica detrás de todo, la vista que cree terminó siendo como una base organizada donde ya estaba reunida toda la información importante: eventos, tipos de evento, creadores, perfiles y participantes y gracias a eso no era necesario repetir todos los JOIN cada vez que se quisiera hacer una consulta, sino simplemente consultar la vista ya creada.

Lo que más me costó entender al principio fue el GROUP BY, no entendía por qué había que agrupar por creador y tipo de evento para después contar los eventos y participantes, pero luego lo entendí como una forma de organizar la información por categorías para que los resultados fueran más claros y fáciles de interpretar.

También comprendí mejor la diferencia entre WHERE y HAVING, ya que antes pensaba que ambos servían para lo mismo, pero con este ejercicio entendí que el WHERE filtra los datos antes de agruparlos, mientras que el HAVING trabaja después del agrupamiento, filtrando ya los resultados obtenidos y esa diferencia me ayudó mucho a entender cómo se procesa una consulta paso a paso.

La parte que más me gustó fue trabajar con PREPARE y EXECUTE, me pareció muy útil poder dejar una consulta preparada y luego ejecutarla varias veces con parámetros diferentes sin tener que escribir todo de nuevo, se siente mucho más práctico y cercano a cómo se trabaja en proyectos reales donde las consultas se reutilizan constantemente.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

En conclusión, este ejercicio me ayudó a entender que una base de datos no solo sirve para guardar información, sino también para consultarla de manera organizada, eficiente y reutilizable.

9.- Informe (calidad de presentación y respeto de los requerimientos)

10.- Participación en Foro de Tareas

11.- Video de Sustentación

12.- Repositorio GIT

<https://github.com/kely-b/bd1-20261-g100-equipo-3/tree/main/Tarea6>